Exam – June 2025

ALG-CPH-F25: Algorithms and Data Structures (SW2)

Instructions. This exam consists of **six questions** and you have **four hours** to solve them. Note that each question begins on a new page. The maximum possible score is **100 points**, but bear in mind that these points are only advisory, and will be weighted following the exam. Please provide your answers digitally, either in a separate document, directly on this exam PDF, or both.

Remember to upload your digital answers as a single PDF file. It will not be possible to hand in anything physically for this exam. Throughout the exam, make sure to clearly indicate which question you are answering.

- Before starting with the exam, please make sure you read and understand these guidelines.
- Also before starting with the exam, *read the entire exam set.* When you have read all the questions, *then read the entire exam set again.* Then, begin answering the questions.
- For each question in the exam, make sure you read the text carefully, and understand what the problem is before solving it. Terms in bold or cursive are of particular importance.
- During the exam, you are only allowed a **limited set of aides**:
  - All hand-written or off-line notes
  - All slides from the course
  - The course book
  - The following online resources:
    - Wikipedia — `https://www.wikipedia.org/`
    - Stack Overflow —- `https://stackoverflow.com/`
    - draw.io (for drawing figures) — `https://app.diagrams.net/`
    - Lucidchart (for drawing figures) — `https://www.lucidchart.com/`
    - Moodle — `https://www.moodle.aau.dk/`
- Specifically, you are **not allowed** to use
  - Search engines (such as Google)
  - Generative AI, neither online nor offline (such as ChatGPT)
  - Tools that directly solve the problem, you are working on (for example a Master Theorem solver)
- You are **not** allowed to communicate with others during the exam, beyond asking for administrative help from the administrative staff.
- Your answers should be short and concise. Make clear references to the book if you use existing algorithms or data structure (for example: 'Then I use Insertion-Sort from CLRS chapter 1 to do something clever')
- You may answer in English, Danish, Norwegian, Swedish, or in any combination of these.

Question 1. 15 Pts

Identifying asymptotic notation. (Note: lg means logarithm in base 2)

(1.1) [5 Pts] Note **ALL** the correct answers: $\lg 3n + 49^{128} + \frac{n}{2}$ is:

a) $\Theta(\lg n)$ b) $\Theta(n)$ c) $\Theta(n \lg n)$ d) $\Theta(n^2)$ e) $\Theta(2^n)$

(1.2) [5 Pts] Note **ALL** the correct answers: $17n - 75000 + \lg n^{2n}$ is:

a) $O(\lg n)$ b) $O(n)$ c) $O(n \lg n)$ d) $O(n^2)$ e) $O(2^n)$

(1.3) [5 Pts] Note **ALL** the correct answers: $0.25n + 2 \lg n + n^{-1}$ is

a) $\Omega(1)$ b) $\Omega(\lg n)$ c) $\Omega(n)$ d) $\Omega(n \lg n)$ e) $\Omega(n^2)$

Question 2. 10 Pts

Solve the following recurrence using the Master Method. Make sure that you show your calculations each step of the way, including which case of the Master Theorem the recurrence matches, and what the asymptotic analysis of the recurrence is. Make sure to simplify your final analysis as much as possible.

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ 7T(n/3) + n^2 & \text{if } n > 1 \end{cases}$$

Question 3. 25 Pts

Understanding of existing algorithms

(3.1) [5 Pts] Note **ALL** the correct answers below:

a) While *insertion* in a red-black tree is easy, *deletion* is problematic and leads to bad performance with many deletions

b) Doubly linked lists support deletion in constant time

c) If $f(n) = \Theta(g(n))$ then $f(n) = \Omega(g(n))$

d) $O(n)$ is a *tight* upper bound for the height of a regular binary search tree with n nodes

e) Assuming independent uniform hashing, the worst case asymptotic runtime for inserting into a hash table using *separate chaining* is $O(n)$

(3.2) [6 Pts] Recall the `Max-Heap Property` and consider the array $A = [16, 9, 10, 15, 8, 7, 12, 1, 14, 2]$

a) At what indicies are the elements in A roots of a proper binary max-heap (ie. satisfying the `Max-Heap Property`)? Assume 1-indexing.

b) At what indicies would you need to call MAX-HEAPIFY in order to convert A into a max-heap?

c) Show the binary max-heap you get from making those calls to MAX-HEAPIFY.

(3.3) [5 Pts] Consider the array $A = [u, e, t, a, u, a, f, s]$

a) Run QUICKSORT on A and show the result after *excactly 2* calls to PARTITION (use alphabetic sorting order)

b) Is the following statement true: 'Assuming a best case scenario, the quicksort algorithm is *asymptotically* faster than the merge sort algorithm'? Explain why or why not.

(3.4) [9 Pts] Hash tables and probe sequences

a) Consider an initially empty hash table T with $m = 8$, a hash-function $m(k * 0.91 - \lfloor k * 0.91 \rfloor)$ and implementated using *chaining*. Show T after inserting the keys 9, 3, 6, 8, 4, 7.

b) Show the probe-sequence for searcing for the key 5 in a hash table with $m = 8$ using double hashing and the auxiliary hash functions $h_1(k) = \lfloor k^2/5 \rfloor$ and $h_2(k) = 4k + 1$

Question 4. 15 Pts

Analysis of new algorithms

Consider the following weird algorithm for sorting a list A:

```
WeirdSort(A)
 1  S = new array of size n
 2  B = new array of size n
 3  while A.size > 0
 4      for i = A.size downto 1
 5          if S.size == 0 or A[i] ≤ S[S.size]
 6              Push(S, Pop(A, i))
 7      Push(B, Pop(S))

 8  while S.size > 0
 9      Push(B, Pop(S))

10  return B
```

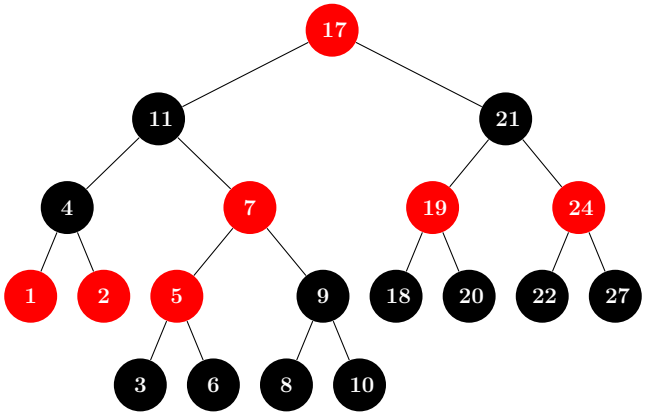
The algorithm WeirdSort works by looking through A (in reverse order) and pushes the smallest element it has seen to the top of a stack S, removing it from A in the process ($\textsc{Pop}(A, i)$ removes and returns the element on index i in A). After each execution of the for-loop, it pops the top element of S and pushes it to the output array B. When A is empty, it pushes the remaining elements of S to B before it returns B:

(4.1) [5 Pts] What is the *best case* input for WeirdSort, and what is the asymptotic running time (in O- or Θ-notation) for this situation? Conversely, what is the *worst case* input and the corresponding running time?

(4.2) [10 Pts] State an appropriate loop invariant for the for-loop in line 4-6 and show that it holds for initialization, maintenance and termination. Also, give a brief argument why your loop invariant is relevant for the correctness of the algorithm.

Question 5. 10 Pts

Consider the following illegal red-black tree T (assume all pictured terminal nodes have two black NIL-children):



(5.1) [5 Pts] List all the ways in which T violates the red-black tree properties (you may refer to a node by the key associated with it).

(5.2) [10 Pts] Perform *two* rotations (LEFT-ROTATE and/or RIGHT-ROTATE) and *one* color-flip so the resulting tree is a valid red-black tree. Note down where you performed the rotations, which node you flipped and show the resulting tree.

Question 6. 25 Pts

This question is about understanding and modelling an algorithmic problem and how to modify an existing algorithm to solve a variation of the problem.

It's May and it's finally time for Eurovision! Once again, you are hosting a party that is sure to turn your friends into true believers of the kitsch, crazy and campy music competition. However, they have not yet learned to truly appreciate the magic of Balkan ballads and pompous staging, so you fear that some of them might not show up unless you make it very easy for them.

Therefore, you decide to calculate the shortest path from each of your friends houses to the place where the big party will go down. For this purpose, you have a map in the form of a graph $G = (V, E)$ with vertices V representing locations (your friends houses, intersections and the party place), edges E representing roads and a weight function $w : E \to \mathbb{R}$ determining how long it takes to travel along a road.

(6.1) [5 Pts] Explain how to find the shortest path from each of your friends houses to your party location in at most $O(VE)$ time.

Thinking about it, you realize that some of your friends might be even more resilient than others to go to your party. So you make a list of each of your friends and assign a score of how bothered you think they would be. The score is a number between 1 and 100 and for every unique road (edge) your friend would have to follow, you will add this number to the length of the path. For example, you know Olga, who is a die-hard metal fan, hates Eurovision, so you assign 80 to her meaning that a path containing 5 roads would have a length of 200 more than in the original map.

(6.2) [10 Pts] Pondering about this for a while, you start to question your idea. Would it work with your previous approach if you modified the algorithm accordingly? Explain why or why not.

After some time, you decide to do something else entirely. Since the party location is somewhat out of town, you decide instead to host the party at one of your friends who lives the closest to everyone else (and you will just bribe that friend with some champagne and a colorful hat!).

(6.3) [10 Pts] Describe how to find that one of your friends who lives the closest to everyone else. What is the run time of your approach?

Thank you for your hard work and active participation in this course!
Remember to upload your answers to the exam as a single PDF file.

Best of luck!
- Andreas